

DSA Problem Patterns — Cheatsheet

A study system, not just a reference — built for actually remembering this, not just reading it once more.

Why This Keeps Slipping (and the actual fix)

Re-reading a cheatsheet feels like learning, but it mostly isn't. Your brain recognizes the pattern the moment it sees it again and mistakes that recognition for knowledge. The gap shows up later, in an interview, when there's no cheatsheet to jog it — because recognition and recall are different skills, and only recall was never practiced.

The fix is **active recall**: cover the answer, force yourself to produce it from memory, then check. It feels harder and slower than reading. That difficulty is the signal it's working — effortless review builds no long-term memory at all.

How to use this document

1. Read pages 2–5 once, slowly, picturing each hook image as you go.
2. Go straight to the Self-Test on the last page. Cover the right-hand column and try to recall the pattern from the clue alone, out loud. Don't skip this — it's the part that actually builds memory.
3. Come back on Day 3, Day 7, and Day 14 and repeat only the Self-Test page (2–3 minutes each time). This spacing is what moves it from short-term to long-term memory — cramming once does not.

The Problem-First Framework

Before reaching for any pattern, ask these four questions in order. This is the actual thought process — the patterns below are just the toolbox this framework points you toward.

1	What's the input shape?	Array / string → often two pointers, sliding window, or binary search. Linked list → fast/slow pointers. Tree / graph / grid → BFS or DFS. A count or optimum over choices → DP.
2	What's actually being asked?	One answer (a value/pair) → search-style pattern. Every possibility (all subsets/paths) → DFS + backtracking. Best/optimal value → DP. Repeated "is X connected to Y" → Union-Find.
3	What do the constraints imply?	$n \leq \sim 20$ → exponential (backtracking) is fine. $n \leq \sim 10,000$ → $O(n^2)$ is fine. $n \leq \sim 10^6$ → needs $O(n)$ or $O(n \log n)$ → two pointers, sliding window, heap, binary search.
4	Is it sorted, or can it be?	Sorted → two pointers or binary search become available. Unsorted, need fast lookup → hash map/set instead.

The 10 Core Patterns

1. Two Pointers

Tell: Sorted array, "pair that sums to X", palindrome check, merging two sorted lists.

Place two pointers — often one at each end of a sorted array — and move them toward each other based on a comparison. Because the array is sorted, moving the left pointer right always increases the sum, and moving the right pointer left always decreases it, letting you eliminate one possibility per step instead of checking every pair.

```
l, r = 0, len(arr)-1
while l < r:
    if arr[l]+arr[r]==target: return [l,r]
    elif arr[l]+arr[r]<target: l += 1
    else: r -= 1
```

Hook: two hands closing in from opposite ends of a sorted bookshelf, meeting in the middle.

Time: $O(n)$ **Space:** $O(1)$

Classic problems: Two Sum II, Valid Palindrome, Container With Most Water, 3Sum

Gotcha: Only works cleanly when sorted (or order doesn't matter). Unsorted + can't sort? Use a hash map instead.

2. Sliding Window

Tell: "Longest/shortest substring or subarray", a contiguous segment plus a condition.

Maintain a window with a left and right boundary. The right pointer expands the window and updates running state; when the state violates your condition, shrink from the left until valid again. Both pointers only move forward, so total work is $O(n)$.

```
l = 0
for r in range(len(arr)):
    # add arr[r] to state
    while window_invalid(state):
        # remove arr[l], l += 1
        l += 1
    best = max(best, r-l+1)
```

Hook: a caterpillar inching along a branch — front foot reaches ahead, back foot catches up only when overstretched.

Time: $O(n)$ **Space:** $O(k)$

Classic problems: Longest Substring Without Repeats, Minimum Window Substring, Max Subarray Sum of Size K

Gotcha: Fixed-size windows just slide (no shrink-loop needed). Variable-size windows need the while-shrink logic.

3. Fast & Slow Pointers

Tell: Linked list; "detect a cycle"; "find the middle node"; "find the cycle's start".

Two pointers walk the same list at different speeds — slow moves one step, fast moves two. If a cycle exists, fast eventually laps slow and they land on the same node. When fast hits the end, slow sits exactly at the midpoint.

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
if slow == fast: return True
```

Hook: a tortoise and a hare on the same looping track — the hare eventually laps the tortoise from behind.

Time: $O(n)$ **Space:** $O(1)$

Classic problems: Linked List Cycle I & II, Middle of the Linked List, Happy Number

Gotcha: Always check `fast and fast.next` in the loop condition or you'll crash on `fast.next.next`.

4. Binary Search (& variants)

Tell: Sorted array — or any "is this answer too big or too small?" question, even for minimize/maximize problems.

Maintain a range $[lo, hi]$ and check the midpoint. Based on the result, discard half the remaining range — each check eliminates half the possibilities, reaching the answer in $O(\log n)$ checks. Works on any monotonic answer space, not just literal sorted arrays.

```
lo, hi = 0, len(arr)-1
while lo <= hi:
    mid = (lo+hi)//2
    if arr[mid]==target: return mid
    elif arr[mid]<target: lo = mid+1
    else: hi = mid-1
```

Hook: the guess-the-number game — "higher" or "lower" halves your remaining guesses every time.

Time: $O(\log n)$ **Space:** $O(1)$

Classic problems: Binary Search, Search in Rotated Sorted Array, Koko Eating Bananas, Ship Packages Within D Days

Gotcha: Off-by-one errors are the #1 bug — fix whether your range is inclusive/exclusive on each end and stay consistent.

5. BFS (Breadth-First Search)

Tell: "Shortest path", "minimum steps/moves", "level order" on a graph, grid, or tree.

Explore outward in expanding rings using a queue, fully finishing each distance before moving to the next. The first time you reach the target, it's guaranteed to be via the shortest path.

```
q = deque([start]); seen = {start}; steps = 0
while q:
    for _ in range(len(q)):
        node = q.popleft()
        if node == target: return steps
        for nxt in neighbors(node):
            if nxt not in seen:
                seen.add(nxt); q.append(nxt)
        steps += 1
```

Hook: drop a stone in a pond — ripples spread outward one ring at a time; the first ripple to touch the target traveled the shortest distance.

Time: $O(V+E)$ **Space:** $O(V)$

Classic problems: Word Ladder, Rotting Oranges, Binary Tree Level Order, Shortest Path in Binary Matrix

Gotcha: Mark nodes as seen the moment you enqueue them, not when you dequeue — otherwise duplicates flood the queue.

6. DFS / Backtracking

Tell: "Generate all combinations/permutations/subsets", "explore every path", mazes, Sudoku, N-Queens.

Go as deep as possible down one branch of choices; on a dead end or full solution, undo the last choice and try the next option at that fork. The undo step is what turns plain DFS into backtracking.

```
def dfs(start, path):
    if is_complete(path):
        results.append(path[:]); return
    for choice in options(start):
        if not valid(choice, path): continue
        path.append(choice)
        dfs(next_start(choice), path)
    path.pop()
```

Hook: a maze runner sprinting down a corridor — hit a wall, walk back to the last fork, try the next door.

Time: $O(2^n)$ / $O(n!)$ **Space:** $O(n)$ depth

Classic problems: Subsets, Permutations, Combination Sum, N-Queens, Word Search

Gotcha: Forgetting `path.pop()` after recursing is the most common bug — it silently corrupts every later branch.

7. Dynamic Programming

Tell: "Minimum/maximum ways to...", "how many distinct ways", overlapping subproblems.

Break into subproblems and store each answer the first time it's computed, so future requests are free lookups instead of recomputation. Only helps when subproblems genuinely overlap. Top-down = recursion + memo; bottom-up = iterative table.

```
memo = {}
def solve(i):
    if i in base_cases: return base_val
    if i in memo: return memo[i]
    memo[i] = combine(solve(i-1), solve(i-2))
    return memo[i]
```

Hook: a student who writes every answer on a cheat sheet the first time they solve it — never re-derives the same sub-question twice.

Time: $O(\text{states})$ **Space:** $O(\text{states})$

Classic problems: Climbing Stairs, Coin Change, Longest Common Subsequence, 0/1 Knapsack, House Robber

Gotcha: The hard part isn't the code — it's finding the recurrence. Ask "what's the smaller version of this exact problem?" first.

8. Monotonic Stack

Tell: "Next greater/smaller element", histogram-area problems, daily temperatures.

Keep a stack that stays strictly increasing (or decreasing). When a new element breaks that order, pop items off — each pop answers a query for the popped element. Every element is pushed once and popped at most once, giving $O(n)$.

```
stack = [] # indices, increasing by value
for i, x in enumerate(arr):
    while stack and arr[stack[-1]] < x:
        j = stack.pop()
        result[j] = i - j
    stack.append(i)
```

Hook: a height-sorted photo line — anyone taller who walks in shoves the shorter people out.

Time: $O(n)$ **Space:** $O(n)$

Classic problems: Daily Temperatures, Next Greater Element, Largest Rectangle in Histogram, Trapping Rain Water

Gotcha: Store indices on the stack, not values — you almost always need the index for a distance or result update.

9. Heap / Priority Queue

Tell: "Kth largest/smallest", "top K elements", merging K sorted lists, scheduling by priority.

A heap keeps the smallest (or largest) element instantly accessible, with $O(\log n)$ insert/remove. For top-K problems: keep a heap of size K, push every candidate, pop the worst if it exceeds K. What's left is exactly the top K.

```
heap = []
for x in nums:
    heapq.heappush(heap, x)
    if len(heap) > k:
        heapq.heappop(heap)
# heap[0] is the kth largest
```

Hook: the VIP line outside a club — the bouncer always knows who's next without re-checking the whole line.

Time: $O(n \log k)$ **Space:** $O(k)$

Classic problems: Kth Largest Element, Top K Frequent Elements, Merge K Sorted Lists, Find Median from Data Stream

Gotcha: Python's heapq is min-heap only — negate values to simulate a max-heap.

10. Union-Find (Disjoint Set)

Tell: "Connected components", "are these in the same group?", cycle in an undirected graph.

Each element starts as its own group. `union(a,b)` merges groups; `find(x)` returns the group's representative — two elements are connected exactly when `find` matches for both. Path compression flattens the structure so repeated queries get faster.

```
parent = list(range(n))
def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]
def union(a, b):
    parent[find(a)] = find(b)
```

Hook: friend clusters at a party — union-find tracks who's clumped with who, and answers "same clump?" almost instantly.

Time: $\sim O(\alpha(n))$ per op **Space:** $O(n)$

Classic problems: Number of Provinces, Redundant Connection, Accounts Merge, Graph Valid Tree

Gotcha: Without path compression, the structure degrades into a chain and operations slow to $O(n)$.

Instant Lookup Table

See this phrase	Reach for	Typical time
sorted array, find pair	Two Pointers	$O(n)$
longest/shortest substring	Sliding Window	$O(n)$
linked list cycle / middle	Fast & Slow Pointers	$O(n)$
sorted, find target / minimize max	Binary Search	$O(\log n)$
shortest path, min steps	BFS	$O(V+E)$
all combinations / permutations	DFS + Backtracking	$O(2^n)$ / $O(n!)$
min/max ways, optimal value	Dynamic Programming	$O(\text{states})$
next greater element	Monotonic Stack	$O(n)$
top K / kth largest	Heap	$O(n \log k)$
same group? / connected?	Union-Find	$\sim O(1)$ per op

Self-Test (the part that actually builds memory)

Fold or cover the right-hand column with a sheet of paper. Read each clue on the left, say the pattern name AND its hook image out loud, then check yourself. Do this page — only this page — again on Day 3, Day 7, and Day 14.

Clue	Pattern + Hook (cover this)
Sorted array, find a pair	Two Pointers — two hands closing in
Longest substring with a condition	Sliding Window — caterpillar inching
Cycle in a linked list	Fast & Slow Pointers — tortoise & hare
Sorted array, find X fast	Binary Search — higher/lower guessing
Shortest path in a graph/grid	BFS — ripples in a pond
All possible combinations	DFS + Backtracking — maze runner
Optimal value, overlapping subproblems	Dynamic Programming — cheat-sheet student
Next greater element in an array	Monotonic Stack — height-sorted line
Top K / Kth largest element	Heap — VIP bouncer
Are these two connected?	Union-Find — friend clumps at a party